UNIVERSITY OF LEEDS

COMP1860 Building our Digital World: Computer Systems & Architecture

S1.W4.L1: Combinational Logic Circuits & Minimisation

Dr James Dyer

Learning Outcomes



UNIVERSITY OF LEEDS

1. Construct a logic circuit using logic gates.
2. Apply minimisation of Boolean expressions to simplify logic circuits

Text Book

- This lecture covers material similar to chapter 12 of **Stallings.** ***Computer Organisation & Architecture***
  - It covers Boolean identities (section 12.1, pages 414 and 415) in greater detail
  - It covers sections 12.3, from Multiplexers onward (page 428)

Overview



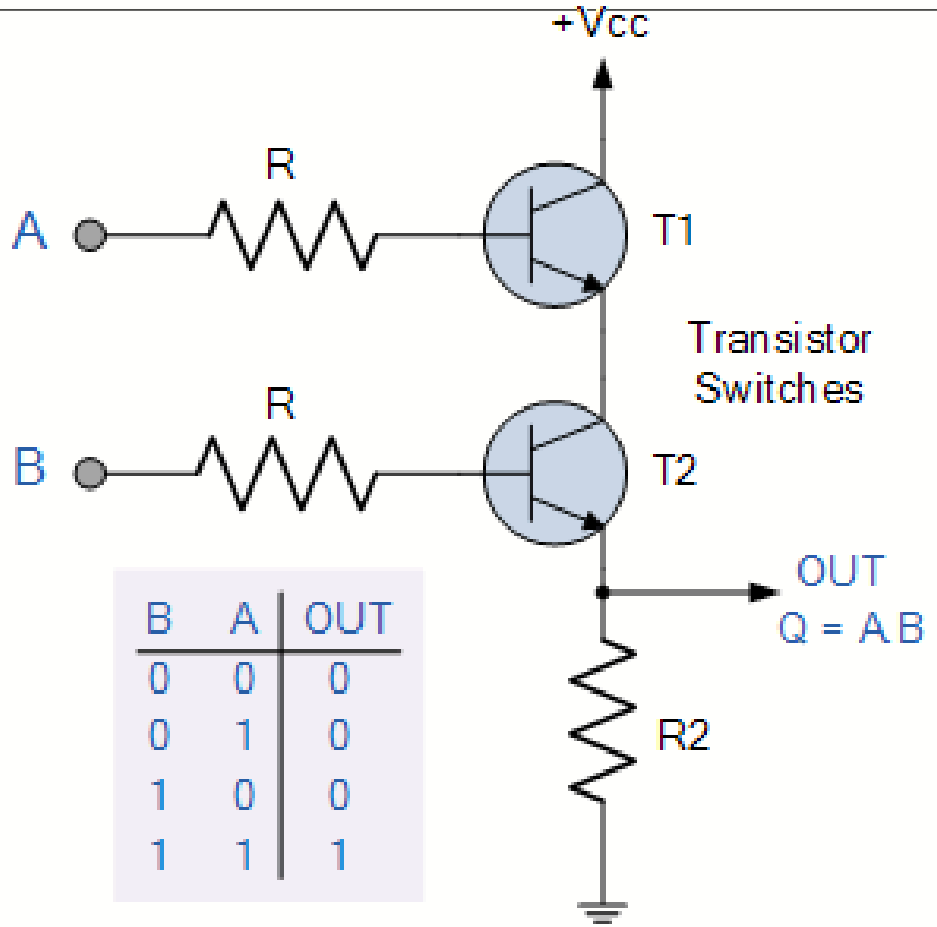
UNIVERSITY OF LEEDS

- Implementing Logic Gates in Electronics
- Combinational Circuits
  - Multiplexer
  - Decoder
  - Adder
- Boolean Algebraic Laws
- Expressive Power of NAND

Implementing Logic Gates in Electronics

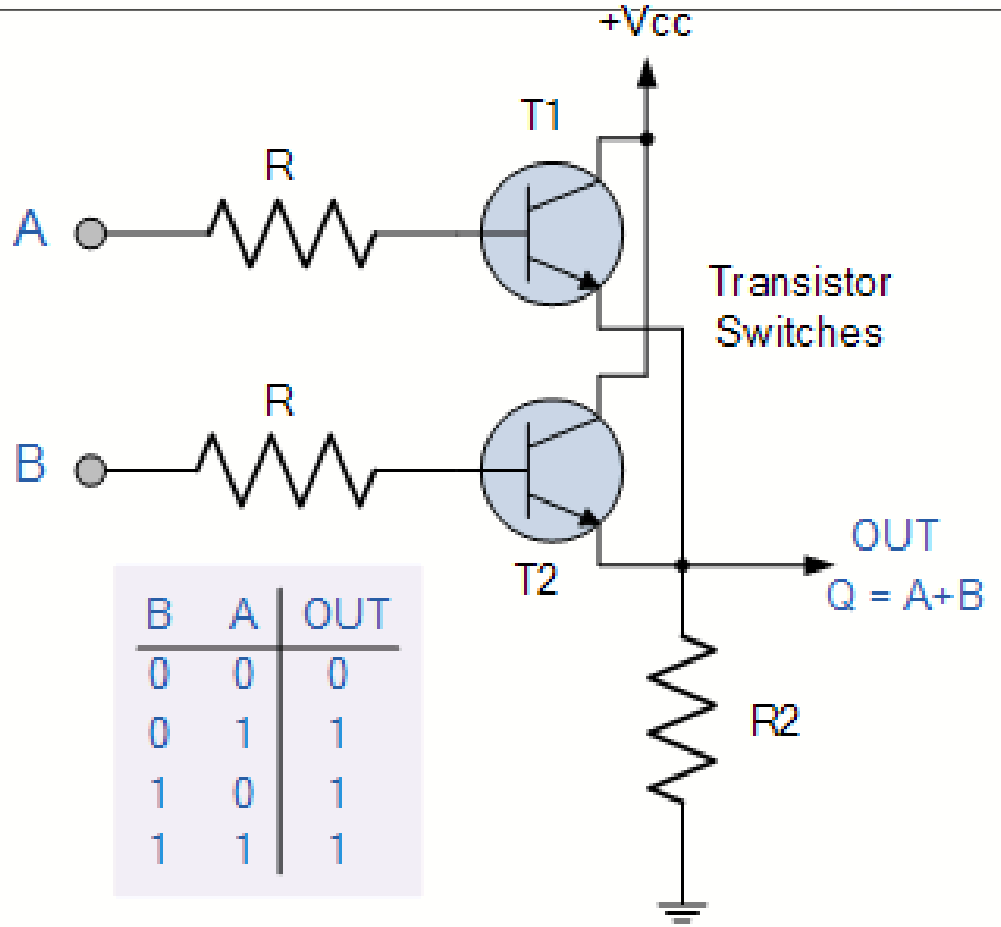
Creating Logic Gates From Transistors(晶体管)

UNIVERSITY OF LEEDS



B	A	OUT
0	0	0
0	1	0
1	0	0
1	1	1

AND Gate

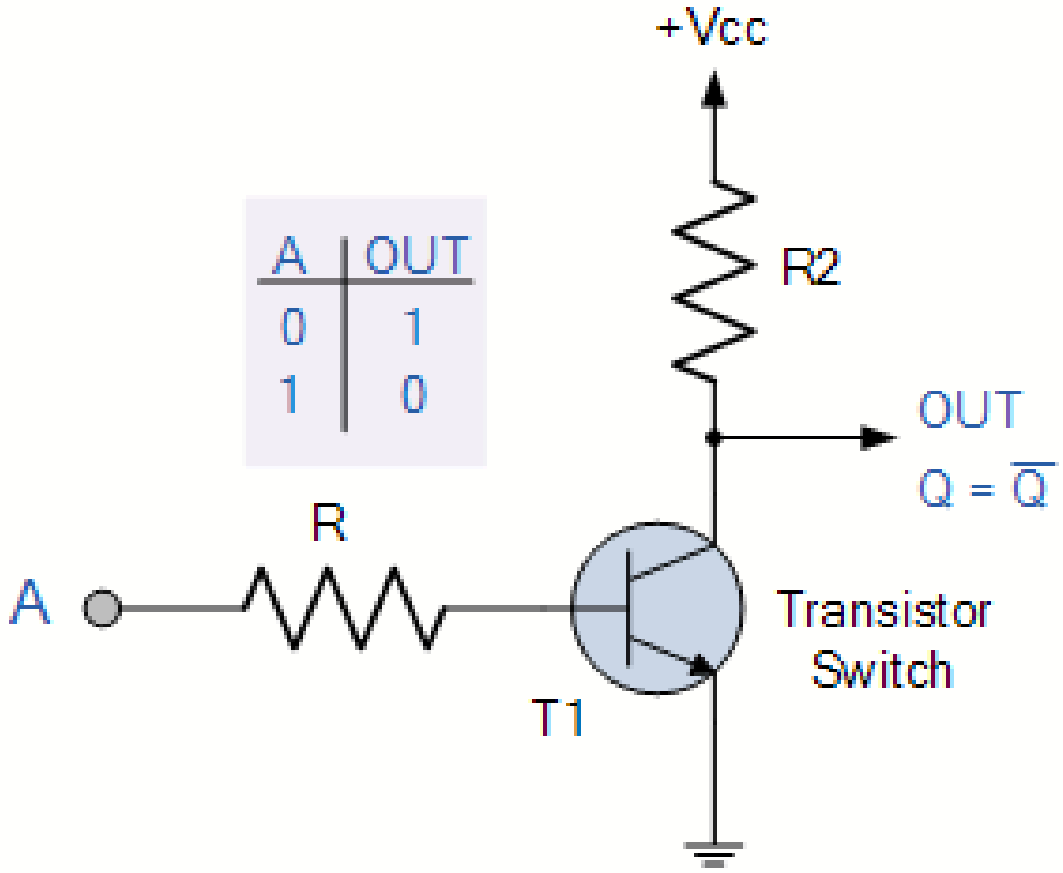


B	A	OUT
0	0	0
0	1	1
1	0	1
1	1	1

OR Gate

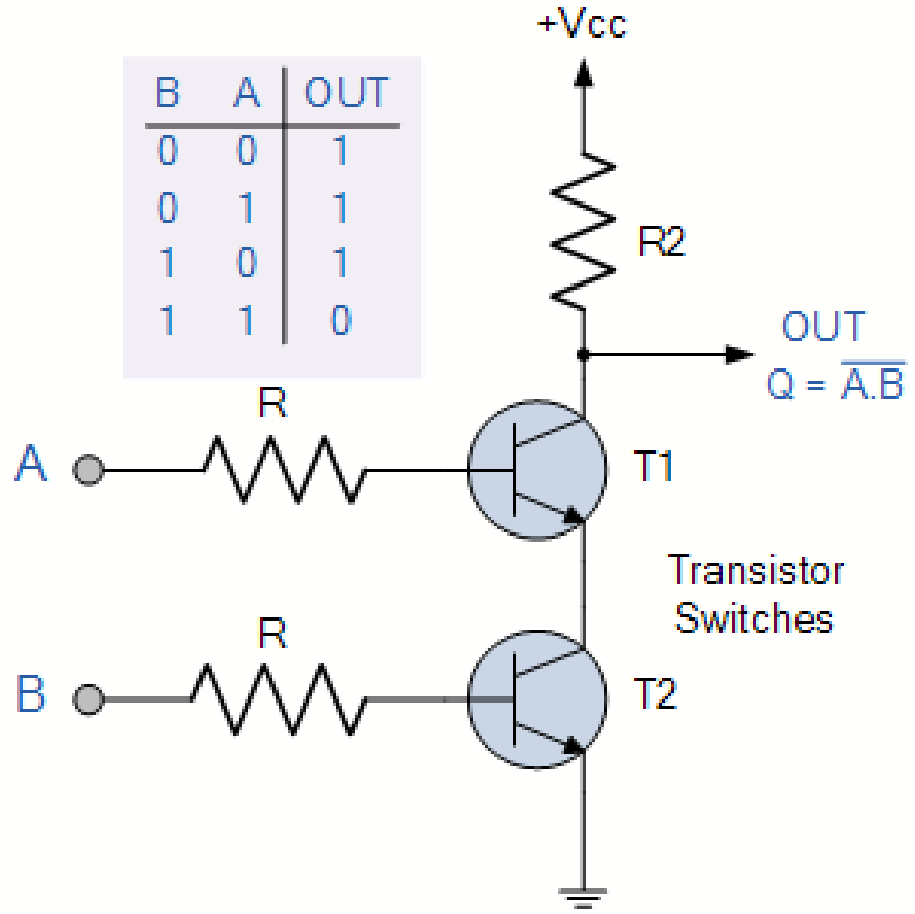
Creating Logic Gates From Transistors （晶体管）

UNIVERSITY OF LEEDS



A	OUT
0	1
1	0

NOT Gate



B	A	OUT
0	0	1
0	1	1
1	0	1
1	1	0

NAND Gate

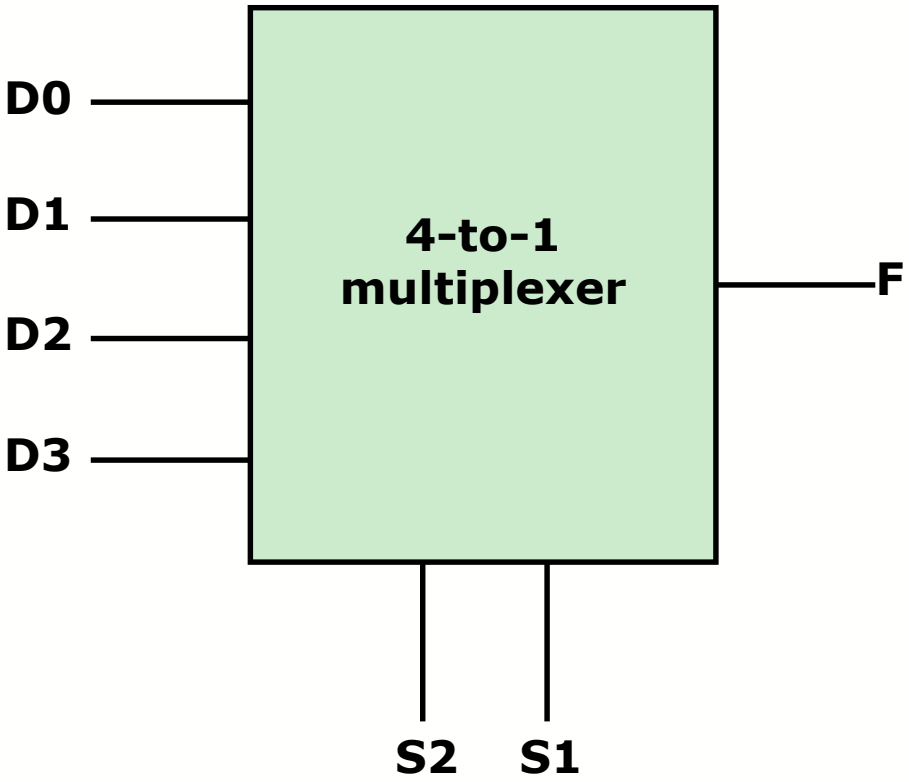
Combinational Circuits

Mutiplexer (信号选择器：MUX，4 in-1 out)

UNIVERSITY OF LEEDS

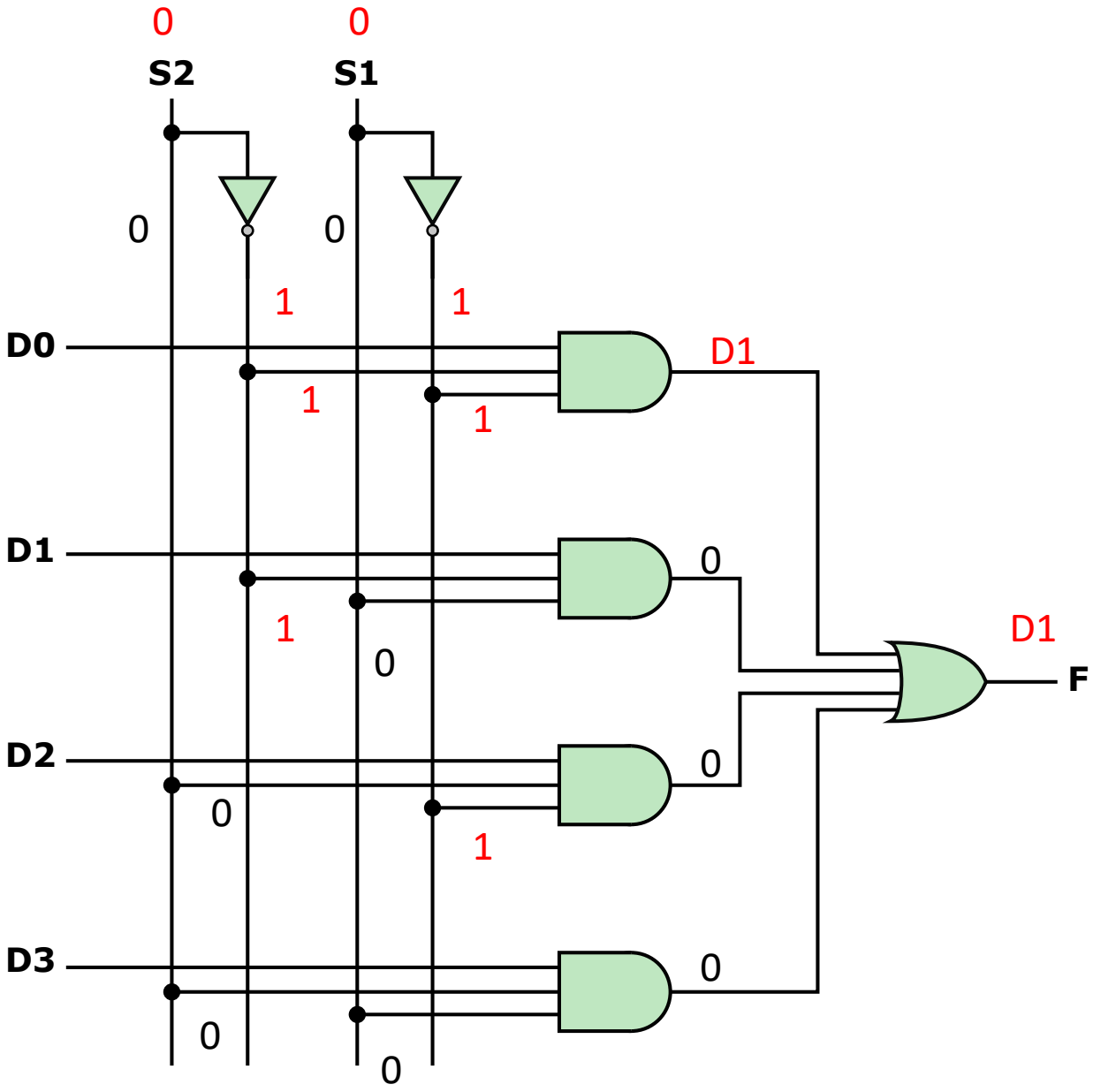
- A multiplexer is a selector(选择器)
- One of the inputs will be output
- Which one depends on the selection inputs (S1, S2, etc.)

S2	S1	F
0	0	D0
0	1	D1
1	0	D2
1	1	D3



选D0-D3哪一路，由S1,S2编码决定，共4种组合(00,01,10,11)

Muliplexer Implementation

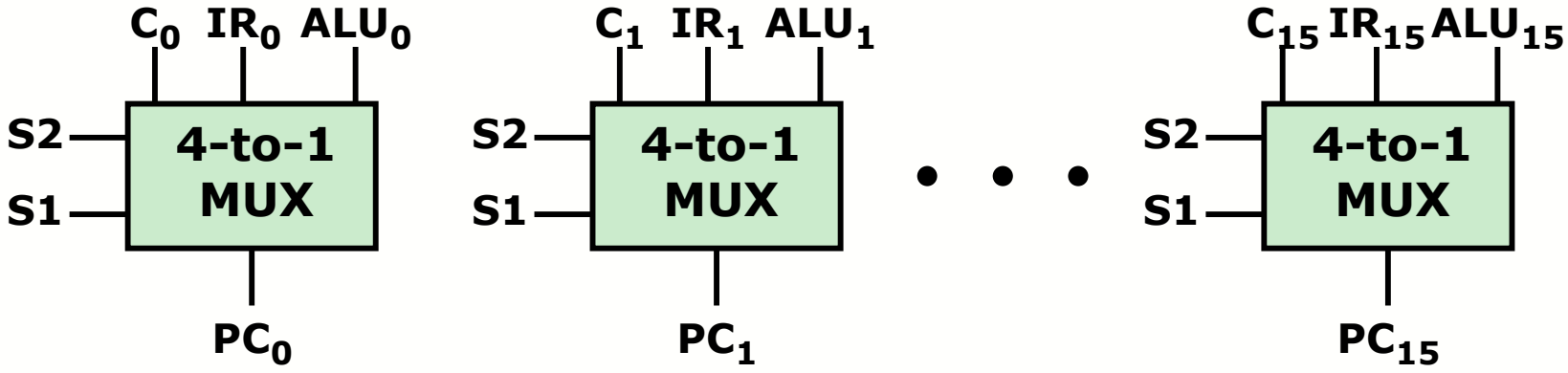


Multiplexer uses

UNIVERSITY OF LEEDS

- Used to control routing(路由) of data and signals
- Example: Program Counter (PC) register
  - Binary value if PC incremented
  - Instruction register if branch instruction executed(执行分支指令)
  - Output from ALU (jumping a number of memory locations to a new address)

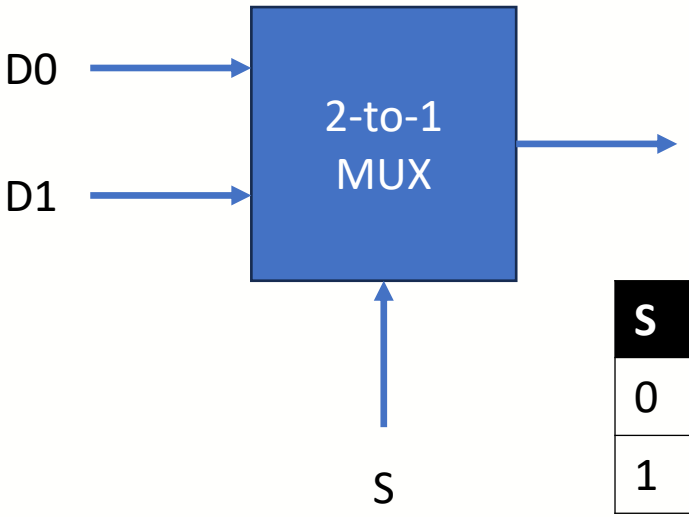
C: PC Counter Register IR: Instruction Register ALU: ALU
通过S1，S2，单个PC register可以切换不同的用途



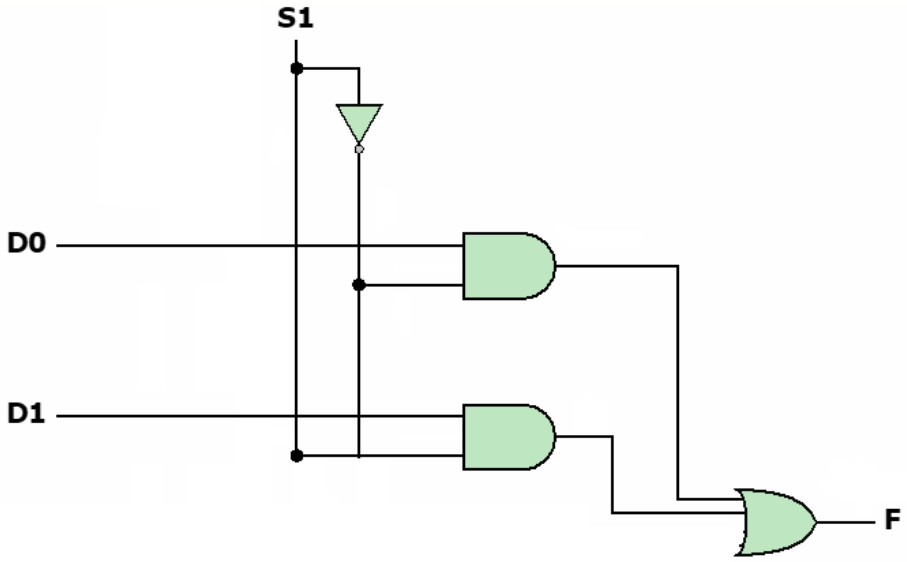
Exercise: 2-to-1 multiplexer

UNIVERSITY OF LEEDS

- Use https://logic.ly/demo or pen and paper to design a 2-to-1 multiplexer
- **Hint**: very similar to slide 50 of last lecture.



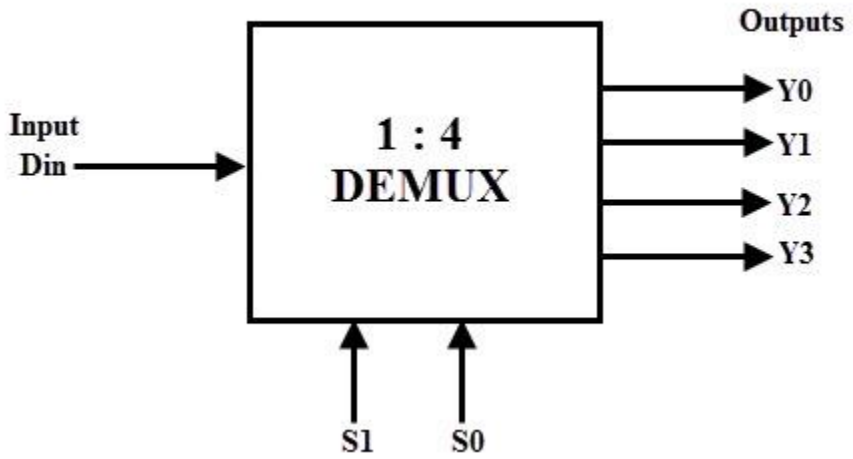
S	F
0	D0
1	D1



Demultiplexer (反向选择器DEMUX：　1in - 4out)

UNIVERSITY OF LEEDS

- A demultiplexer does the opposite
- Selects output path
  - S inputs determine which output line the input is sent to



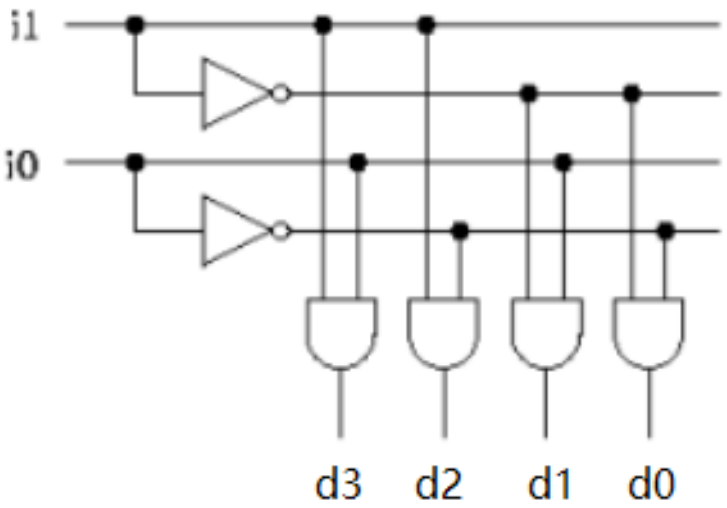
Multiplexer：　input 4选1，　输出到output(4IN-1OUT)
Demultiplexer:　output 4选1，输出到output(1IN-4OUT)

Decoder（解码器 n in – 2^n out）

UNIVERSITY OF LEEDS

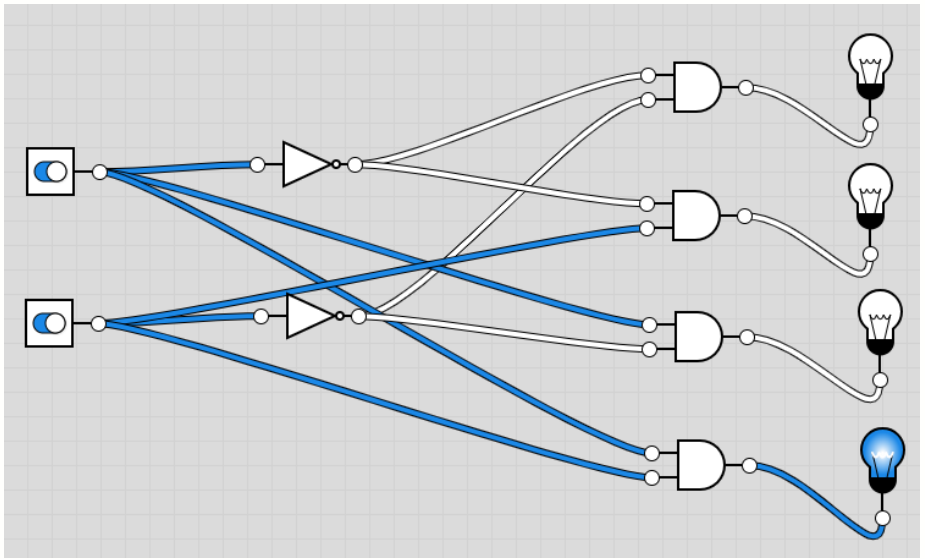
- A circuit with:
  - *n* inputs - 2^n outputs
  - i0i1组成的2进制码,可看做1的左移位数, 用来选择d3-d0中的一个, 其中00 : d0, 01 : d1, 10 : d2, 11 : d3
- Determines output line based on inputs

mux & decoder & selector详解



i_1	i_0	d_3	d_2	d_1	d_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

2-to-4 decoder

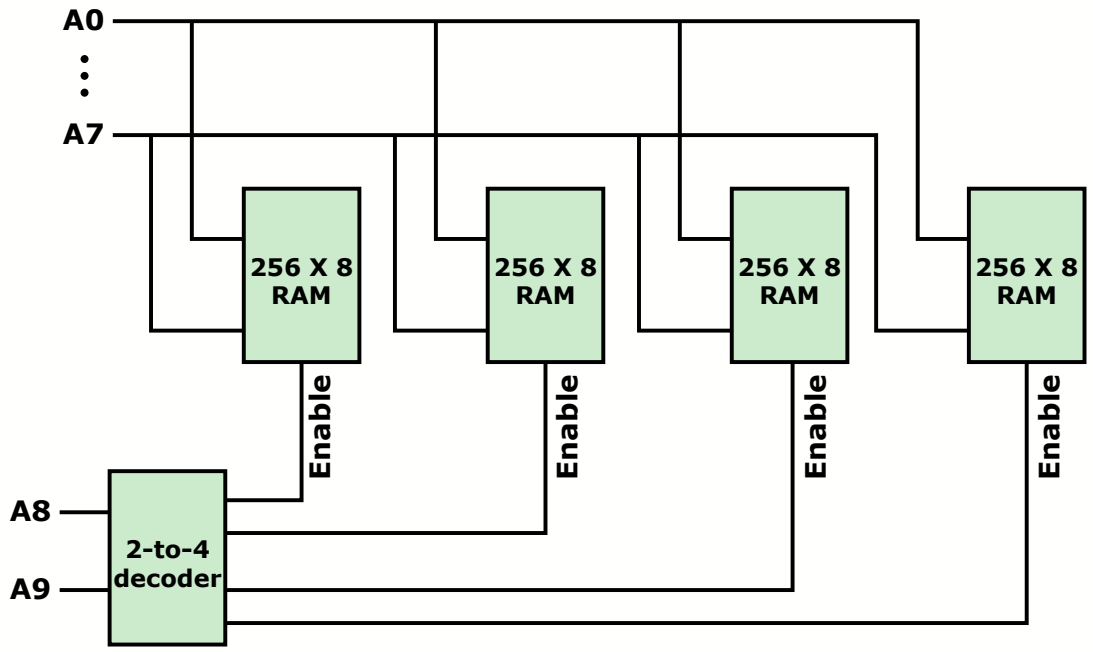


Decoder Use

UNIVERSITY OF LEEDS

- Example: Address Decoding（地址解码）
- Address Space

Address	Chip
0000 – 00FF	0
0100 – 01FF	1
0200 – 02FF	2
0300 – 03FF	3



- 10-bit address
  - 2 bits to select chip（2选4 decoder）
  - 8 bits to select location in chip（8选256 decoder）
  - 有4个内存条

Exercise: 1-to-2 decoder



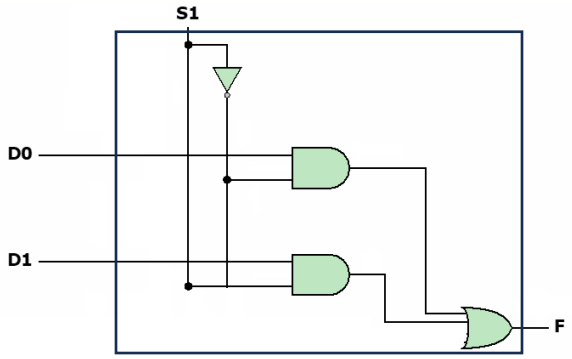
UNIVERSITY OF LEEDS

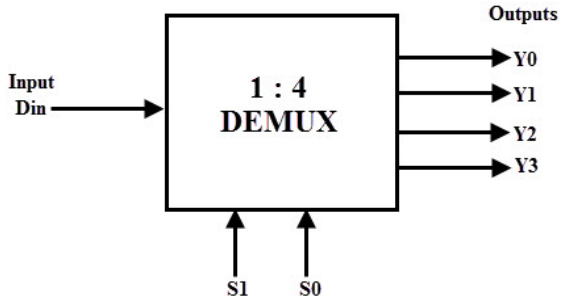
- Use https://logic.ly/demo or pen and paper to design a 1-to-2 decoder
  - If input is 0, switch on bulb 1, turn off bulb 2
  - If input is 1, turn off bulb 1, switch on bulb 2

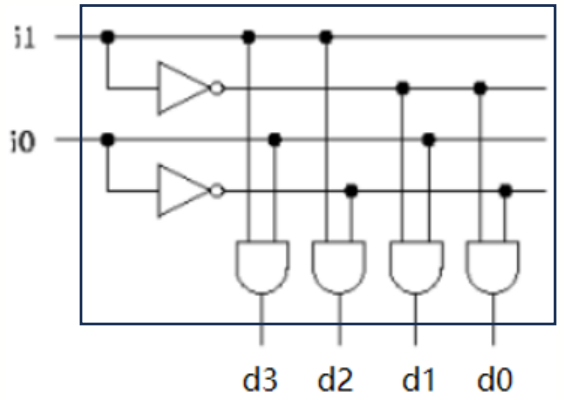
MUX, DEMUX, Decoder对比

UNIVERSITY OF LEEDS

	MUX	DEMUX	Decoder
INPUT/OUTPUT	n IN - 1 OUT	1 IN - N OUT	n IN – 2^n OUT
Use	control routing(路由) of data and signals Eg PC register	Selects output path	Address Decoding 地址解码







Adder（累加器）

UNIVERSITY OF LEEDS

- An adder is a circuit which adds two bit and outputs the sum and carry bits（进位）.
- Remember binary addition
  - 0 + 0 = 0
  - 0 + 1 = 1
  - 1 + 0 = 1
  - 1 + 1 = 10 (value 0 plus carry 1)

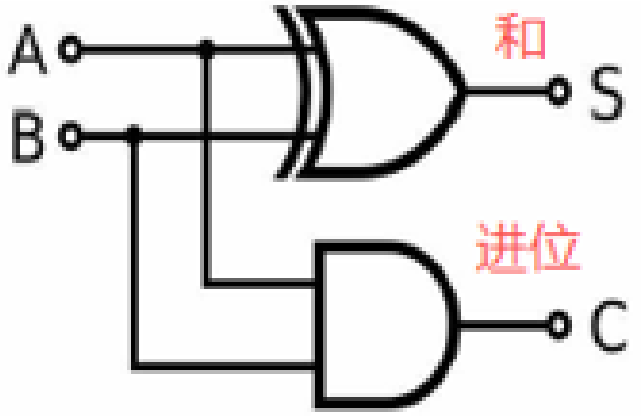
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Exercise: 1-bit Half Adder（1位半加器）

UNIVERSITY OF LEEDS

- Using https://logic.ly/demo or pen and paper, design a circuit which implements this truth table:

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



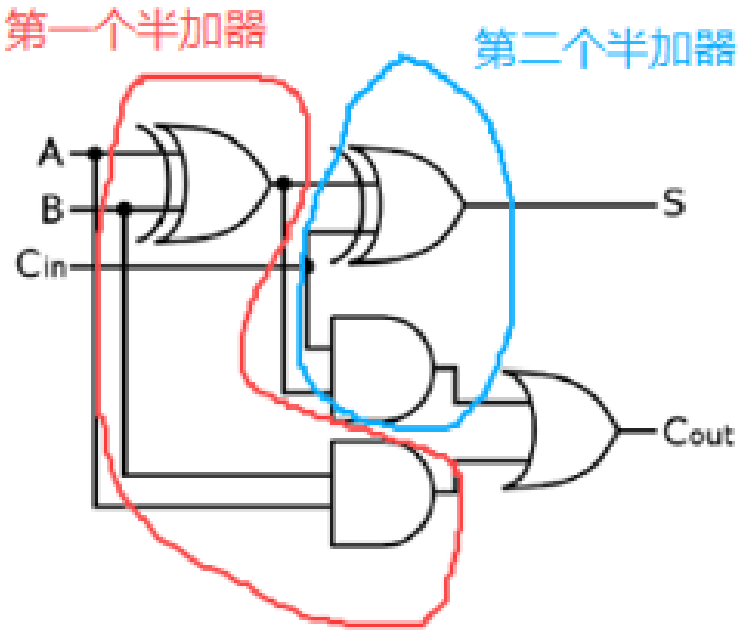
- It should have 2 inputs, A and B, and 2 outputs, Sum and Carry.
- Hint: look at the truth tables for Sum and Carry. Which logical operators are they the same as?

Full 1-bit Adder（1位全加器）

UNIVERSITY OF LEEDS

- This should have 3 inputs
  - A
  - B
  - CarryIn（低位的进位）
- 2 outputs
  - Sum
  - CarryOut

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	1	0	1	0
1	0	0	1	0
1	1	0	0	1
0	0	1	1	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	1

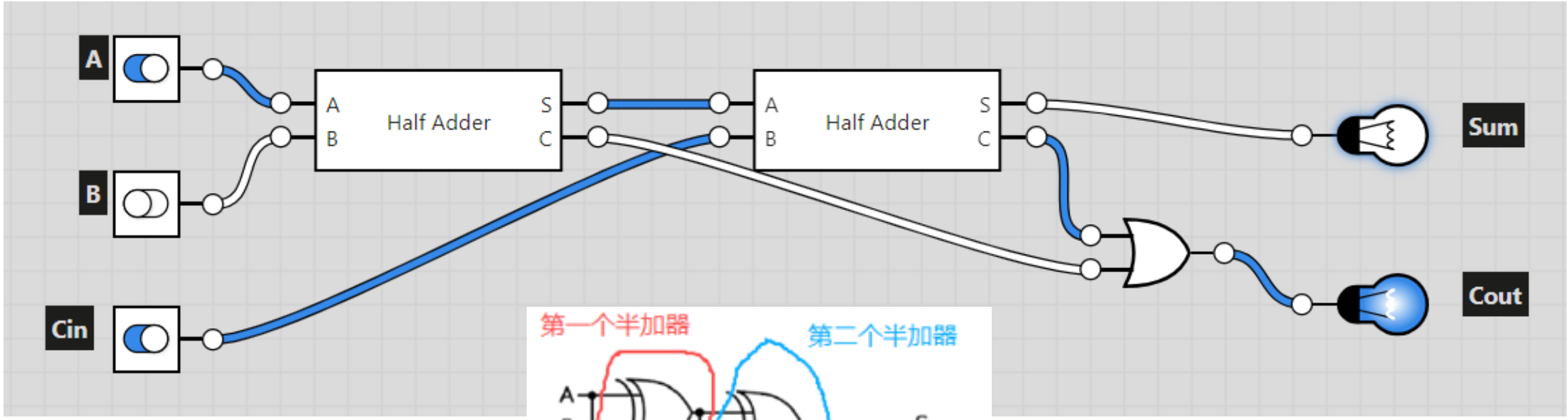


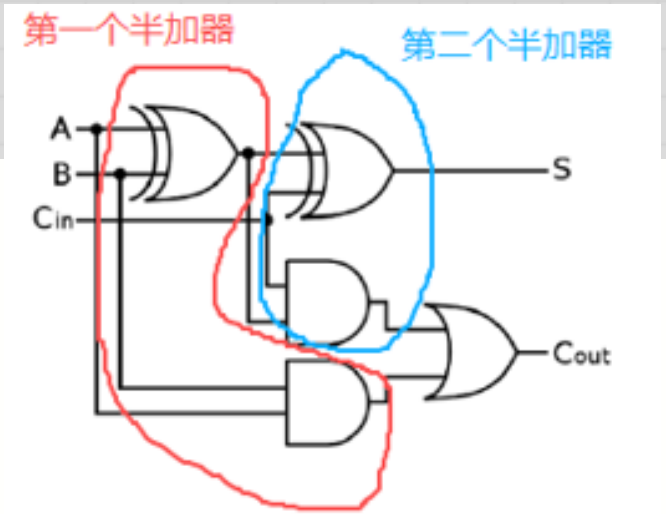
全加器比半加器多了一个input：来自低位的进位input

Full Adder Circuit

UNIVERSITY OF LEEDS

- A full adder can be implemented with two half adders.

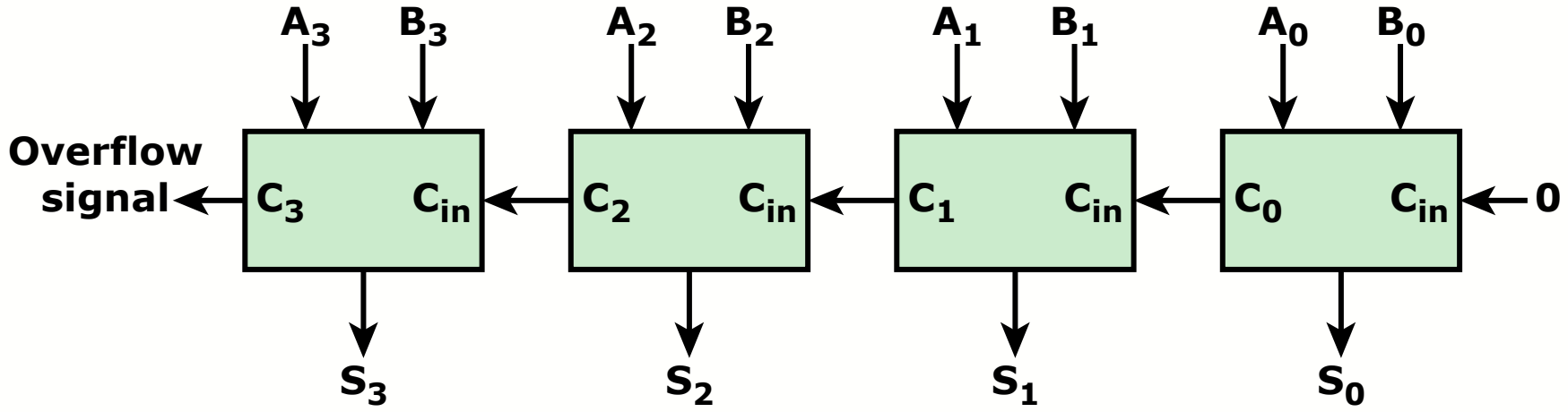




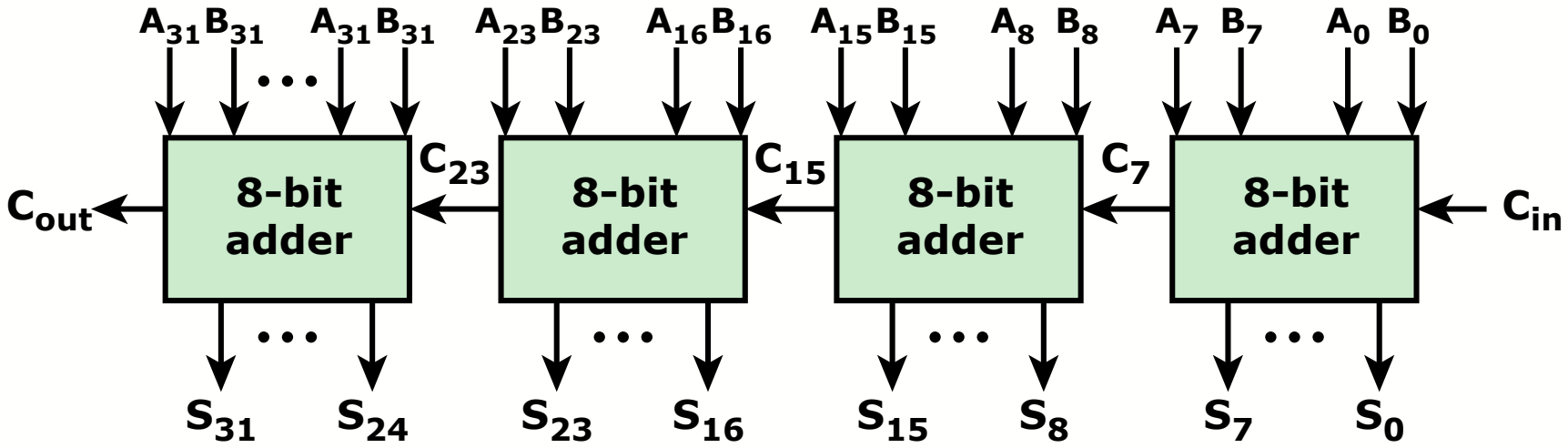
Multibit adders（多位全加器）

UNIVERSITY OF LEEDS

4-bit adder built from 1-bit full adders



32-bit adder built from 8-bit full adders



Boolean Algebraic Laws

Boolean Algebraic Laws



UNIVERSITY OF LEEDS

Name	AND Form	OR Form
Identity Law	$1 \wedge A = A$	$0 \vee A = A$
Null Law	$0 \wedge A = 0$	$1 \vee A = 1$
Idempotent Law	$A \wedge A = A$	$A \vee A = A$
Inverse Law	$A \wedge \neg A = 0$	$A \vee \neg A = 1$
Commutative Law	$A \wedge B = B \wedge A$	$A \vee B = B \vee A$
Associative Law	$(A \wedge B) \wedge C = A \wedge (B \wedge C)$	$(A \vee B) \vee C = A \vee (B \vee C)$
Distributive Law	$A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$	$A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$
Absorption Law	$A \wedge (A \vee B) = A$	$A \vee (A \wedge B) = A$
De Morgan's Law	$\neg(A \wedge \mathrm{B}) = \neg A \vee \neg B$	$\neg(A \vee B) = \neg A \wedge \neg B$

Identity Law

UNIVERSITY OF LEEDS

AND Form

- Truth table:

1	A	$\mathbf{1} \wedge \boldsymbol{A}$
1	0	0
1	1	1

- $1 \wedge A = A$

OR Form

- Truth table:

0	A	$0 \vee \boldsymbol{A}$
0	0	0
0	1	1

- $0 \vee A = A$

Null Law

UNIVERSITY OF LEEDS

AND Form

- Truth table:

0	A	$\mathbf{0} \wedge \boldsymbol{A}$
0	0	0
0	1	0

- $0 \wedge A = 0$

OR Form

- Truth table:

0	A	$1 \vee \boldsymbol{A}$
1	0	1
1	1	1

- $1 \vee A = 1$

Idempotent Law

UNIVERSITY OF LEEDS

AND Form

- Truth table:

A	A	$A \wedge A$
0	0	0
1	1	1

- $A \wedge A = A$

OR Form

- Truth table:

A	A	$A \vee A$
0	0	0
1	1	1

- $A \vee A = A$

Inverse Law

UNIVERSITY OF LEEDS

AND Form

- Truth table:

A	$\neg A$	$A \wedge \neg A$
0	1	0
1	0	0

- $A \wedge \neg A = 0$

OR Form

- Truth table:

A	$\neg A$	$A \vee \neg A$
0	1	1
1	0	1

- $A \vee \neg A = 1$

Commutative Law

UNIVERSITY OF LEEDS

AND Form

- Truth table:

A	B	$A \wedge B$	$B \wedge A$
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

- $A \wedge B = B \wedge A$

OR Form

- Truth table:

A	B	$A \vee B$	$B \vee A$
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1

- $A \vee B = B \vee A$

Associative Law

UNIVERSITY OF LEEDS

AND Form

- Truth table:

A	B	C	$(A \wedge B) \wedge C$	$A \wedge (B \wedge C)$
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

- $(A \wedge B) \wedge \mathrm{C} = A \wedge (B \wedge C)$

OR Form

- Truth table:

A	B	C	$(A \vee B) \vee C$	$A \vee (B \vee C)$
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

- $(A \vee B) \vee \mathrm{C} = A \vee (B \vee C)$

Distributive Law

UNIVERSITY OF LEEDS

AND Form

- Truth table:

A	B	C	$A \vee (B \wedge C)$	$(A \vee B) \wedge (A \vee C)$
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

- $A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$

OR Form

- Truth table:

A	B	C	$A \wedge (B \vee C)$	$(A \wedge B) \vee (A \wedge C)$
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

- $A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$

Absorption Law

UNIVERSITY OF LEEDS

AND Form

- Truth table:

A	B	$A \wedge (A \vee B)$
0	0	0
0	1	0
1	0	1
1	1	1

- $A \wedge (A \vee B) = A$

OR Form

- Truth table:

A	B	$A \vee (A \wedge B)$
0	0	0
0	1	0
1	0	1
1	1	1

- $A \vee (A \wedge B) = A$

De Morgan's Law

UNIVERSITY OF LEEDS

AND Form

- Truth table:

A	B	$\neg(A \wedge B)$	$\neg A \vee \neg B$
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

- $\neg(A \wedge B) = \neg A \vee \neg B$

OR Form

- Truth table:

A	B	$\neg(A \vee B)$	$\neg A \wedge \neg B$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

- $\neg(A \vee B) = \neg A \wedge \neg B$

Example: simplifying Boolean expressions



UNIVERSITY OF LEEDS

- $A \wedge \neg(A \vee B)$
- $\equiv A \wedge (\neg A \wedge \neg B)$ (De Morgan's Law)
- $\equiv (A \wedge \neg A) \wedge \neg B$ (Associative Law)
- $\equiv 0 \wedge \neg B$ (Inverse Law)
- $\equiv 0$ (Null Law)

Example: simplifying Boolean expressions



UNIVERSITY OF LEEDS

- $(A \wedge B) \vee (B \wedge C)$
- $\equiv (A \vee B) \wedge (A \vee C) \wedge (B \vee B) \wedge (B \vee C)$ (Distributive Law)
- $\equiv (A \vee B) \wedge (A \vee C) \wedge B \wedge (B \vee C)$ (Idempotent Law)
- $\equiv (A \vee B) \wedge (A \vee C) \wedge B$ (Absorption Law)
- $\equiv (A \vee C) \wedge B \wedge (A \vee B)$ (Commutative Law)
- $\equiv (A \vee C) \wedge B \wedge (B \vee A)$ (Commutative Law)
- $\equiv (A \vee C) \wedge B$ (Absorption Law)

Examples: Simplifying Boolean Expressions



UNIVERSITY OF LEEDS

- $\mathrm{F} = (\neg A \wedge \neg B \wedge C) \vee \neg((A \wedge B) \vee (A \wedge C))$
- 2nd term: $\neg\big((A \wedge B) \vee (A \wedge C)\big) = \neg(A \wedge B) \wedge \neg(A \wedge C)$ (De Morgan's Law)
- $= (\neg A \vee \neg B) \wedge (\neg A \vee \neg C)$ (De Morgan's Law)
- $= \big(\neg A \wedge (\neg A \vee \neg C)\big) \vee \big(\neg B \wedge (\neg A \vee \neg C)\big)$ (Distributive Law)
- $= (\neg A \wedge \neg A) \vee (\neg A \wedge \neg C) \vee (\neg B \wedge \neg A) \vee (\neg B \wedge \neg C)$ (Distributive Law)
- $= \neg A \vee (\neg A \wedge \neg C) \vee (\neg B \wedge \neg A) \vee (\neg B \wedge \neg C)$ (Idempotent Law)
- $= \neg A \vee (\neg A \wedge \neg B) \vee (\neg B \wedge \neg C)$ (Absorption & Commutative Laws)
- $= \neg A \vee (\neg B \wedge \neg C)$ (Absorption Law)
- Therefore, $\mathrm{F} = (\neg A \wedge \neg B \wedge C) \vee \neg A \vee (\neg B \wedge \neg C)$
- $\mathrm{F} = \neg A \vee \big(\neg A \wedge (\neg B \wedge C)\big) \vee (\neg B \wedge \neg C)$ (Commutative and Associative Laws)
- $\mathrm{F} = \neg A \vee (\neg B \wedge \neg \mathrm{C})$ (Absorption Law)

Exercise

- Simplify the following expressions using Boolean algebraic laws:

1. $\mathrm{F} = A \vee \neg(B \wedge A)$
2. $\mathrm{F} = (A \vee B) \wedge (A \vee C)$

The Expressive Power of NAND

The Expressive Power of NAND（与非门）

UNIVERSITY OF LEEDS

A	B	$A \overline{\wedge} B$
0	0	1
0	1	1
1	0	1
1	1	0

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	1

A	$\neg A$
0	1
1	0

- Any logical operator can be expressed using NANDs only
- NAND = AB先and，再not = $\overline{A \wedge B}$

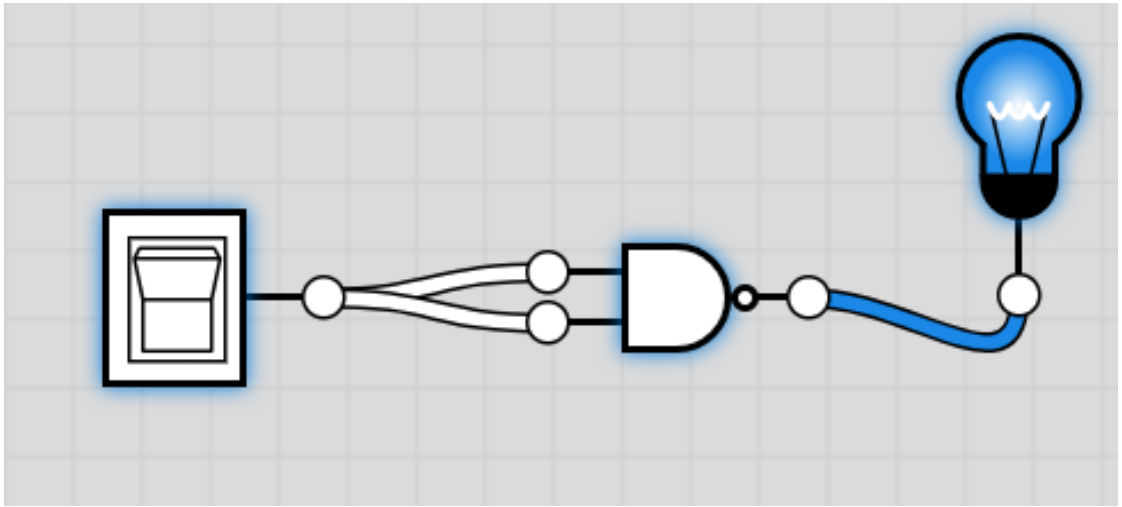
Example: NOT

UNIVERSITY OF LEEDS

- $\bar{A} \equiv \overline{A \wedge A} \equiv \mathrm{A} \mathbin{\overline{\wedge}} \mathrm{A}$

A	A	$A \mathbin{\overline{\wedge}} A$
0	0	1
1	1	0

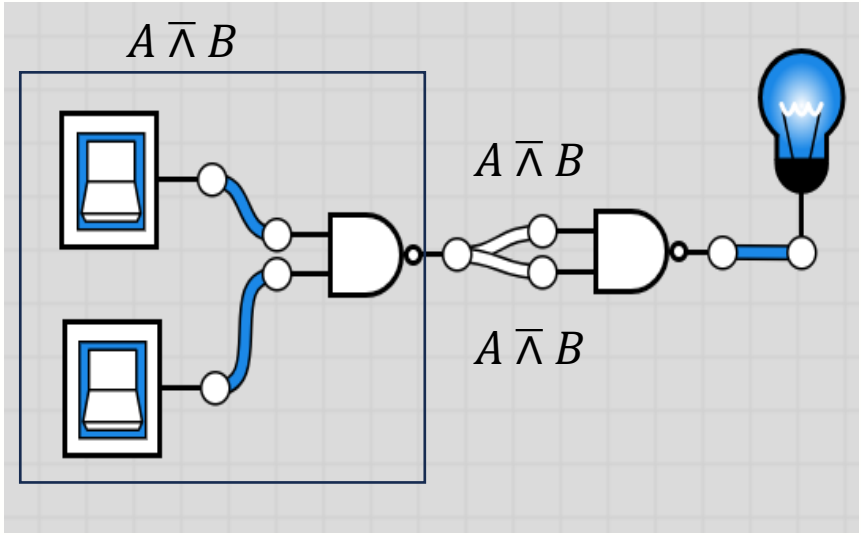
A	$\neg A$
0	1
1	0



Example: AND $(A \wedge B \equiv (A \barwedge B) \barwedge (A \barwedge B))$

UNIVERSITY OF LEEDS

- $A \barwedge B \equiv \overline{A \wedge B}$
- Therefore, $A \wedge B \equiv \neg(\overline{A \wedge B})$
- But NOT is not a NAND??!!
- Remember from previous slide $\neg A \equiv A \barwedge A$
- Hence, $A \wedge B \equiv \neg(\overline{A \wedge B}) \equiv (\overline{A \wedge B}) \barwedge (\overline{A \wedge B}) \equiv (A \barwedge B) \barwedge (A \barwedge B)$

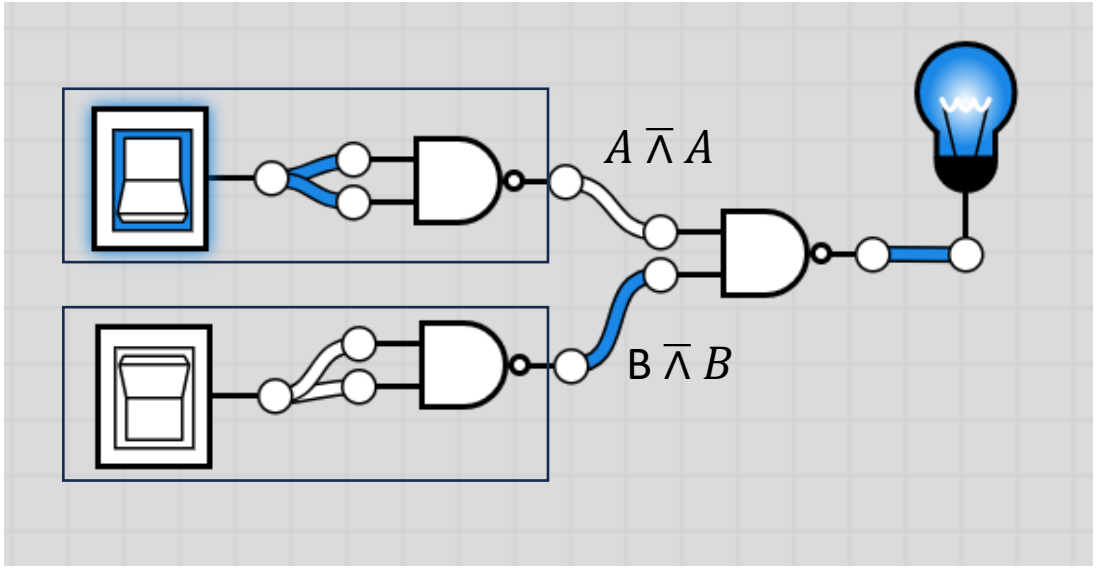


- 意义：用2个与非门来实现： $A \wedge B$

Example: OR $(A \vee B \equiv (A \barwedge A) \barwedge (B \barwedge B))$

UNIVERSITY OF LEEDS

- $\neg(A \vee B) \equiv \neg A \wedge \neg B$ (DeMorgan's Law)
- Therefore $A \vee B \equiv \neg(\neg A \wedge \neg B)$
- $\equiv \neg\big(\neg(\neg A \barwedge \neg B)\big)$ (Remember, $A \barwedge B \equiv \neg(A \wedge B)$
- $\equiv \neg A \barwedge \neg B$
- $\equiv (A \barwedge A) \barwedge (B \barwedge B)$ (Remember, $\neg A \equiv A \barwedge A$)



- 意义：用3个与非门来实现： $A \vee B$

Example: XOR($A \oplus B \equiv (A \barwedge (A \barwedge B)) \barwedge (B \barwedge (A \barwedge B))$)

UNIVERSITY OF LEEDS

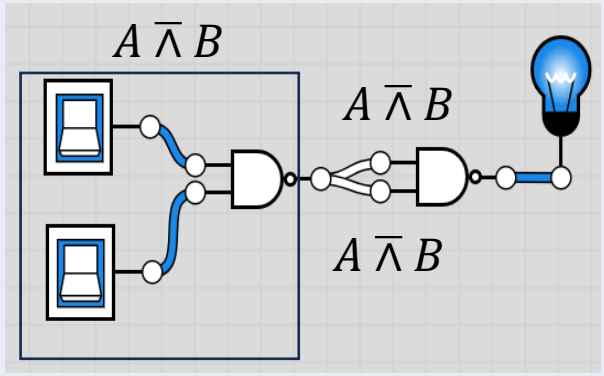
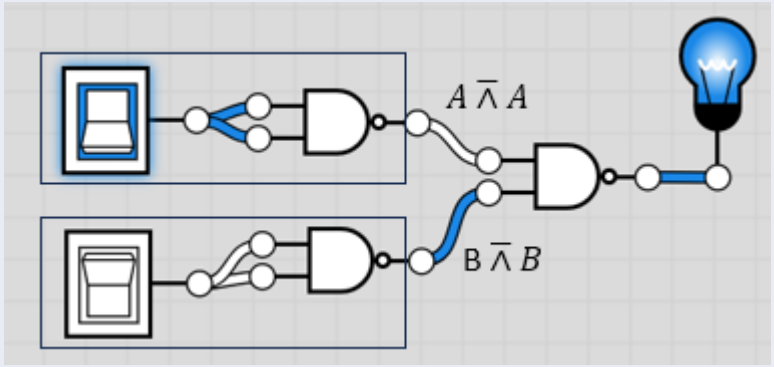
- Observe that $A \oplus B \equiv (A \vee B) \wedge (A \barwedge B)$
- $\equiv (A \wedge (A \barwedge B)) \vee (B \wedge (A \barwedge B))$ (分配率)
- $\equiv \neg(A \barwedge (A \barwedge B)) \vee \neg(B \barwedge (A \barwedge B))$
- $\equiv \neg((A \barwedge (A \barwedge B)) \wedge (B \barwedge (A \barwedge B)))$ (De Morgan's Law)
- $\equiv (A \barwedge (A \barwedge B)) \barwedge (B \barwedge (A \barwedge B))$
- 意义：用4个与非门来实现XOR

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

A	B	$A \vee B$	$\wedge$	$A \barwedge B$
0	0	0	0	1
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

总结：用与非门NAND来实现其他门电路

UNIVERSITY OF LEEDS

$A \wedge B$	$A \vee B$	$A \oplus B$
$A \wedge B \equiv (A \barwedge B) \barwedge (A \barwedge B)$	$A \vee B \equiv (A \barwedge A) \barwedge (B \barwedge B))$	$A \oplus B \equiv \big(A \barwedge (A \barwedge B)\big) \barwedge \big(B \barwedge (A \barwedge B)\big)$
		

Exercise

UNIVERSITY OF LEEDS

1. Implement a circuit for XOR using only NAND gates by translating the expression from the previous slide:

$$A \oplus B \equiv \left(A \barwedge (A \barwedge B)\right) \barwedge \left(B \barwedge (A \barwedge B)\right)$$

2. Create logical circuits for the following using only NAND gates:

a) NOR ($\overline{\vee}$) ($A \mathbin{\overline{\vee}} B \equiv \neg(A \vee B)$)

$$\equiv \neg\left((A \barwedge A) \barwedge (B \barwedge B)\right) \equiv (A \barwedge A) \wedge (B \barwedge B)$$

a) XNOR ($\odot$) ($A \odot B \equiv \neg(A \oplus B)$)

$$\equiv \neg\left(A \barwedge (A \barwedge B)\right) \barwedge \left(B \barwedge (A \barwedge B)\right) \equiv \left(A \barwedge (A \barwedge B)\right) \wedge \left(B \barwedge (A \barwedge B)\right)$$

Summary



UNIVERSITY OF LEEDS

- Implementing Logic Gates in Electronics
- Combinational Circuits
  - Multiplexer
  - Decoder
  - Adder
- Boolean Algebraic Laws
- Expressive Power of NAND